

PP Lecture 6 PDC

Part 1: Past Papers

MIDTERM1 SESSIONAL-I EXAM FALL 2024 • QUESTION 1

Q: For the task dependency graph given below (Tasks 1–5 with weight 10 each, Task 6 with weight 15, Task 7 with weight 5, Task 8 with weight 12, Task 9 with weight 8), find the following parameters:

- a) Maximum degree of concurrency
- b) Total Amount of Work
- c) Critical Path Length
- d) Average degree of concurrency
- e) Furthermore, map the task dependency graph on five processes, namely P_a , P_b , P_c , P_d , and P_e , to reduce the interaction overhead.

A:

- a) Maximum degree of concurrency = 5
- b) Total Amount of Work = $10 + 10 + 10 + 10 + 10 + 5 + 15 + 12 + 8 = 90$
- c) Critical Path Length = $10 + 15 + 12 + 8 = 45$
- d) Average degree of concurrency = $90 / 45 = 2.0$
- e) Optimal Mapping to 5 processes:
 - Map Tasks 1 and 9 on P_a
 - Map Tasks 2 and 8 on P_b
 - Map Tasks 3 and 7 on P_c
 - Map Tasks 4 and 6 on P_d
 - Map Task 5 on P_e

MIDTERM1 SESSIONAL-I EXAM FALL 2024 • QUESTION 2

Q: From Flex, we want to enlist the students enrolled in the CS or DS program of FAST School of Computing in the Lahore campus of NUCES. For the query given below, make a simple task dependency graph. Also confirm how many tasks will be performed for the complete execution of the query:

CAMPUS = "LAHORE" AND SCHOOL = "COMPUTING" AND (PROGRAM = "CS" OR PROGRAM = "DS")

A: The query computation decomposes into relational selection/filter tasks:

- Task 1: Filter records with CAMPUS = "LAHORE"
- Task 2: Filter records with SCHOOL = "COMPUTING"
- Task 3: Filter records with PROGRAM = "CS"
- Task 4: Filter records with PROGRAM = "DS"
- Task 5: Compute (PROGRAM = "CS" OR PROGRAM = "DS") using outputs of Tasks 3 and 4
- Task 6: Compute (CAMPUS = "LAHORE" AND SCHOOL = "COMPUTING") using outputs of Tasks 1 and 2
- Task 7: Compute final AND intersection between outputs of Tasks 5 and 6

Total tasks performed = 7 tasks.

Q: Recall how merge sort works.

- a) Using the example data, provide a recursive decomposition.
- b) Using the same data as above, provide a data-based decomposition.
- c) In terms of potential for better concurrency (and better speedup), which one of the above two is a better decomposition approach?
- d) Provide rationale for your above answer logically and succinctly.

A:

- a) Recursive decomposition repeatedly divides the sequence into half until single-element subsequences remain, forming a recursive task dependency tree where merging occurs bottom-up.
- b) Data-based decomposition partitions the input dataset upfront into contiguous chunks, each task sorts its chunk independently, and a final task combines the sorted results.
- c) Data-based decomposition.
- d) Rationale: In recursive decomposition, there is significant inter-task interaction and overhead during pairwise merging steps. Data-based decomposition divides data upfront among processes so they execute sorting independently without interaction until the final combine step.

Q:

- a) Consider the problem of computing the product $y = Ab$, where A is a sparse 8×8 matrix and b is a dense 8×1 vector. Create a task interaction graph for this problem, such that the i -th task computes the entry of $y[i]$ and owns the i -th row of A and the element $b[i]$.
- b) Map the tasks created in part (a) to four processes such that two consecutive tasks are assigned to one process. Calculate the list C_i (where $i = 1$ to 4) which contains the indices of b that the tasks on Process i need to access from tasks mapped to other processes.

A:

- a) The Task Interaction Graph (TIG) consists of 8 nodes (Tasks 1 to 8). An undirected edge connects Task i and Task j whenever $A[i,j] \neq 0$ or $A[j,i] \neq 0$, indicating that computing $y[i]$ requires element $b[j]$ owned by Task j .
- b) Process allocation (2 tasks per process):
 - Process 1 (Tasks 1, 2): $C_1 = (3, 4, 5)$
 - Process 2 (Tasks 3, 4): $C_2 = (1, 2, 6)$
 - Process 3 (Tasks 5, 6): $C_3 = (1, 2, 3, 4, 8)$
 - Process 4 (Tasks 7, 8): $C_4 = (5)$

SESSIONAL-I EXAM SPRING 2025 • QUESTION 5

Q: Examine the task-dependence graph and compute:

- (a) Maximum degree of concurrency
- (b) Critical Path Length
- (c) Average degree of concurrency
- (d) Minimum number of processors needed to obtain the maximum possible speedup
- (e) Maximum speedup if number of processors are limited to 4

A:

- (a) Maximum degree of concurrency = 5
- (b) Critical Path Length = $15 + 8 + 15 + 12 = 50$
- (c) Average degree of concurrency = Total Work / Critical Path Length = 1.98
- (d) Minimum number of processors needed for maximum speedup = 5
- (e) Maximum speedup limited to 4 processors = $13 / 5 = 2.6$

SPRING 2024 PDC MIDTERM EXAM • QUESTION 6

Q: For the task dependency graph given (Root 34; Level 1: 12, 9, 7, 21, 8; Level 2: 4, 5, 9; Finish: 3), determine the critical path length, maximum degree of concurrency, and average degree of concurrency. Furthermore, map the task graph to three processes to minimize the idling time of the application.

A:

Maximum degree of concurrency = 5 (5 concurrent tasks available at Level 1).

Critical Path Length = $34 + 21 + 9 + 3 = 67$ (path through heaviest weights).

Average degree of concurrency = Total Work / Critical Path Length.

Mapping to 3 processes: Assign independent branches evenly across P_0, P_1, P_2 so that processes executing critical path nodes finish without waiting for delayed dependencies.

QUIZ 01-B SPRING 2023 • QUESTION 7

Q: Choose the correct option:

The _____ of a task-_____ graph is usually a _____ of the _____ of the task-dependency graph:

- a) edge-set; interaction; superset; nodes
- b) edge-set; mapping; subset; edge-set
- c) edge-set; interaction; superset; edge-set
- d) edge-set; interaction; subset; edge-set

A: c) edge-set; interaction; superset; edge-set

QUIZ 01-B SPRING 2023 • QUESTION 8

Q: Analyze the following task-dependency graph (Task 1: 10, Task 2: 10, Task 3: 10, Task 4: 10, Task 5: 6, Task 6: 11, Task 7: 7) and calculate:

- (i) The maximum degree of concurrency**
- (ii) Critical Path Length**
- (iii) The average degree of concurrency**

A:

- (i) Maximum degree of concurrency = 4
- (ii) Critical Path Length = $10 + 6 + 11 + 7 = 34$
- (iii) Average degree of concurrency = Total Work / Critical Path Length = $(10 + 10 + 10 + 10 + 6 + 11 + 7) / 34 = 64 / 34 = 1.88$

QUIZ 01 SPRING 2022 • QUESTION 9

Q: We discussed two different metrics for concurrency and looked at two similar examples of task-dependency graphs. What were those two different metrics, and why was it that in those two graphs, one of the metrics was the same, and the other was different for both?

A:

Maximum Degree of Concurrency: The maximum number of tasks that can be executed simultaneously in a parallel program at any given time (for trees, equal to the number of leaves).

Average Degree of Concurrency: The ratio of total amount of work to critical path length.

Explanation: In both task graphs, the maximum degree of concurrency was the same (4) because both trees had 4 leaf nodes. However, the average degree of concurrency differed (2.33 vs 1.88) because the second graph had a longer critical path length (34 vs 27) for approximately the same total work (64 vs 63).

GENERAL EXAM PAPER / SOLUTION.PDF • QUESTION 10

Q: Multiple Choice Questions on Parallel Algorithm Design:

- i. In case of _____ decomposition, divide and conquer strategy is used.
- ii. The number of tasks that can be executed in parallel is called _____.
- iii. For task dependency graphs that are trees, the maximum degree of concurrency is:
- iv. The use of _____ decomposition comes into play when we _____:

A:

- i. a) Recursive
- ii. b) degree of concurrency
- iii. c) Equal to the number of leaves in the tree
- iv. b) exploratory; are not sure of search space size a priori

GENERAL EXAM PAPER / SOLUTION.PDF • QUESTION 11

Q: Differentiate between Static and Dynamic task mapping.

A:

Static Task Mapping: Used when the size and number of tasks are known a priori. Tasks are assigned to processes before program execution begins.

Dynamic Task Mapping: Used when the size and number of tasks are not known a priori (or generated dynamically at runtime). Tasks are mapped to processes dynamically during execution to maintain load balance.

Part 2: Book Questions

INTRODUCTION TO PARALLEL COMPUTING, CHAPTER 3 • QUESTION 1

Q: In Example 3.2, each union and intersection operation can be performed in time proportional to the sum of the number of records in the two input tables. Based on this, construct the weighted task-dependency graphs corresponding to Figures 3.2 and 3.3. What is the average degree of concurrency of each graph?

A: Node weights are defined by the record counts processed at each relational query node. Critical path length is the sum of node weights along the longest directed path. For Figure 3.2, Critical Path Length = 27, Total Work = 63, so Average Concurrency = $63 / 27 = 2.33$. For Figure 3.3, Critical Path Length = 34, Total Work = 64, so Average Concurrency = $64 / 34 = 1.88$.

INTRODUCTION TO PARALLEL COMPUTING, CHAPTER 3 • QUESTION 2

Q: For the task graphs given in Figure 3.42, determine the following:

- Maximum degree of concurrency.
- Critical path length.
- Maximum achievable speedup over one process assuming an arbitrarily large number of processes is available.
- Minimum number of processes needed to obtain maximum possible speedup.
- Maximum achievable speedup if the number of processes is limited to (a) 2, (b) 4, and (c) 8.

A:

Maximum degree of concurrency: Peak count of tasks executable simultaneously across all time steps.

Critical path length (l): Weight sum of nodes on the longest path from start to finish node.

Maximum speedup $S_{\max} = W / l$, where W is total work.

Minimum processes for $S_{\max}$: Smallest process count required to process all concurrent tasks without extending l.

Speedup under limit p: Calculated via level-by-level task scheduling with p processes.

INTRODUCTION TO PARALLEL COMPUTING, CHAPTER 3 • QUESTION 3

Q: What are the average degrees of concurrency and critical-path lengths of task-dependency graphs corresponding to the decompositions for matrix multiplication shown in Figures 3.10 and 3.11?

A: For fine-grained row/column decompositions with n^2 tasks of unit weight: Total Work $W = O(n^3)$ operations, Critical Path Length $l = O(\log n)$ (with tree reduction) or $O(n)$. Average degree of concurrency = W / l .

INTRODUCTION TO PARALLEL COMPUTING, CHAPTER 3 • QUESTION 4

Q: Let d be the maximum degree of concurrency in a task-dependency graph with t tasks and a critical-path length l. Prove that $d \geq \lceil t/l \rceil$.

A: Proof: The critical path length l defines the minimum number of time steps required to complete all t tasks. By the Pigeonhole Principle, distributing t tasks across at least l time steps requires at least one time step to contain $\geq \lceil t/l \rceil$ tasks executing concurrently. Since d is the maximum number of concurrent tasks at any single step, $d \geq \lceil t/l \rceil$.

INTRODUCTION TO PARALLEL COMPUTING, CHAPTER 3 • QUESTION 5 & 6

Q: Consider LU factorization of a dense matrix split into 9 blocks. Draw a task-dependency graph and enumerate the critical paths.

A:

The TDG begins with Task 1 (factor $A_{1,1}$), which unlocks Tasks 2, 3, 4, 5 (row/column panel updates). These tasks unlock Tasks 6–9 (outer-product updates of trailing submatrices). Task 6 unlocks Task 10 (factor $A_{2,2}$), which unlocks Tasks 11, 12, leading to Task 13, and finally Task 14 (factor $A_{3,3}$).

Critical paths follow the sequence: Task 1 → Task 2 (or 3/4/5) → Task 6 → Task 10 → Task 11 (or 12) → Task 13 → Task 14.

INTRODUCTION TO PARALLEL COMPUTING, CHAPTER 3 • QUESTION 7, 8, & 9

Q: Show an efficient mapping of the LU factorization task-dependency graph onto three processes vs four processes. Which solves the problem faster?

A:

3 Processes: Map block rows or block columns cyclically (Process i owns row/col i). This achieves balanced work distribution during each major step of LU elimination and keeps dependent panel update communications between adjacent processes.

4 Processes: Use a 2D block-cyclic mapping onto a 2×2 process grid. Task $A_{i,j}$ is mapped to Process $(i \bmod 2, j \bmod 2)$.

Comparison: The mapping onto 3 processes solves the problem faster because 3 divides the 3×3 block grid dimensions evenly, resulting in better load balance and fewer idle processes compared to 4 processes.

INTRODUCTION TO PARALLEL COMPUTING, CHAPTER 3 • QUESTION 10, 11, 12, & 13

Q: For an $m \times m$ partitioning of LU factorization, derive an expression for the number of tasks $t(m)$, the maximum degree of concurrency $d(m)$, and the critical-path length $l(m)$.

A:

$t(m)$: The recurrence is $t(m) = t(m-1) + m^2$. Solving yields $t(m) = (m^3 + 2m^2 - m) / 3 = O(m^3/3)$.

$d(m)$: $d(m) = (m-1)^2$, corresponding to the outer-product trailing submatrix updates during step 1.

$l(m)$: $l(m) = 3m - 2$ under unit task execution costs.

INTRODUCTION TO PARALLEL COMPUTING, CHAPTER 3 • QUESTION 15, 16, & 17

Q: In an FFT-like pattern (Algorithm 3.4), assume a decomposition where each execution of the inner loop is a task. Describe the task-dependency and interaction graphs. How would you map $n = 16$ to 16 processes and 8 processes?

A:

Graphs: Task Dependency Graph: $m = \log_2 n$ stages; tasks at stage j depend on outputs of stage $j-1$ tasks.

Task Interaction Graph: Task i interacts with Task $i \oplus 2^j$ at stage j , forming a hypercube interaction graph.

16 Processes: Map Task i directly to Process i . Communication follows 4D hypercube channels.

8 Processes: Map 2 tasks per process (e.g., Process k gets Tasks k and $k+8$), collapsing hypercube communication along dimension 4 to local memory reads.

Q: Consider multiplying two $n \times n$ dense matrices. Compare row-wise data decomposition vs 2D block decomposition. Which decomposition leads to a better parallel algorithm?

A: 2D block decomposition leads to a better parallel algorithm. It reduces the communication-to-computation ratio to $O(1/\sqrt{p})$ compared to $O(1)$ for 1D row-wise decomposition, offering significantly higher scalability.

Q: Compute the best- and worst-case speedup for a centralized scheme for dynamic mapping of seven tasks (1, 2, 3, 4, 5, 5, 10 units) with two processes. What are the key performance trade-offs of centralized dynamic mapping?

A:

Total work $W = 30$.

Best-case schedule: P_1 gets (10, 5) = 15; P_2 gets (5, 4, 3, 2, 1) = 15. Execution time $T_p = 15$. Best-case Speedup = $30 / 15 = 2.0$.

Worst-case schedule: P_1 gets (10, 5, 5) = 20; P_2 gets (4, 3, 2, 1) = 10. Execution time $T_p = 20$. Worst-case Speedup = $30 / 20 = 1.5$.

Trade-offs: Dynamic mapping reduces process idling when task execution times are variable or unpredictable. However, the central master process becomes a contention bottleneck as process count p increases.